CampusHub

AI-Powered Smart Campus Marketplace & Student Collaboration Platform

Complete Product Requirements Document
A Beginner-Friendly, Step-by-Step Execution Guide for a 7-Member Team

This edition rewrites the original PRD so that **every member — regardless of prior experience — knows exactly what to install, what to build, in what order, how to test it, and how to explain it in the viva.** Every section is written in plain language first, then followed by the technical detail.

Primary domain	Generative AI / Agentic AI
Supporting domains	NLP • Computer Vision • Machine Learning • Cloud Computing • Network Security
Team size	7 members, one owner per subsystem
Timeline	14 weeks, milestone-based
Document purpose	Single source of truth for build, test, deploy and viva

How to Use This Document

This PRD is organized so that a member can read only the parts relevant to them and still understand the whole system. Read in this order:

1. **Part 1** — read this first, everyone, once. It explains the product in plain language: what it does, who it's for, and how the pieces fit together. No code, no jargon left unexplained.
2. **Parts 2–5** — the shared technical reference: requirements, user journeys, architecture, database, APIs, and how each AI feature actually works. Skim once, then return to the relevant sub-section whenever you build a feature that touches it.
3. **Part 6** — find your member number (M1–M7) and read your own chapter closely. It contains your installs, your exact weekly tasks, your Definition of Done, your tests, and your viva answers.
4. **Parts 7–12** — shared team practices (Git, environment variables, testing, security, deployment, timeline). Read once as a team before Week 1, then use as a checklist.
5. **Part 13** — full viva preparation with model answers for all seven roles. Read your own section thoroughly; skim the others so you can support teammates in a group viva.

How to work with this document day to day: Every numbered task list in this document is a literal to-do list. When a task is finished, check it off on the GitHub Project Board (Part 12) and attach evidence (screenshot, test output, or short note) as described in Part 11.6.

Document Map

Section	Contents	Primary Audience
Part 1	Understanding CampusHub in Plain Language	Everyone
Part 2	Requirements — Functional & Non-Functional	Everyone, especially M1, M3, M7
Part 3	User Journeys — Full Step-by-Step Walkthroughs	Everyone
Part 4	System Architecture, Database & API Contract	M1, M3, M7 (reference for all)
Part 5	How Every AI/ML Feature Works	M4, M5, M6 (reference for all)
Part 6	Team Structure & Individual Execution Plans (M1–M7)	Everyone — find your chapter
Part 7	Shared Team Practices — Git, Environments, Accounts	Everyone
Part 8	Testing Strategy	M7 leads, everyone contributes
Part 9	Security Requirements	M7 leads, everyone follows
Part 10	Deployment Plan	M7 leads, M1/M3 support
Part 11	14-Week Timeline, Weekly Routine & Evidence Log	Everyone
Part 12	Final Deliverables & Acceptance Checklist	M1 coordinates, everyone contributes
Part 13	Viva Preparation — Questions and Model Answers	Everyone, by role

PART 1

Understanding CampusHub

What we are building, why it matters, and how the pieces fit together — in plain language

1.1 What CampusHub Is, in One Paragraph

Plain-English summary: CampusHub is a mobile app, built only for verified students of one college (or a small group of colleges), where a student can buy and sell used items with other students, chat with a buyer or seller, and get help from AI along the way — AI writes the listing for you, lets you search by typing a normal sentence instead of exact keywords, looks at your product photo to guess its category, suggests a fair price, recommends items you'll probably like, and answers basic questions about the app and campus rules.

It is not a general marketplace like OLX or Facebook Marketplace. It is deliberately smaller and safer: every user is a verified student, every listing is campus-relevant, and every AI feature has a narrow, well-defined job instead of being an open-ended chatbot bolted on top.

1.2 The Problem We Are Solving

Right now, students trade textbooks, calculators, cycles, hostel furniture and electronics through scattered WhatsApp groups, word of mouth, or public marketplaces that were never designed for a student audience. This creates three concrete problems:

- **No trust signal.** A stranger on a public marketplace could be anyone. A CampusHub user is a verified student of a known college.
- **No structure.** A WhatsApp group cannot be searched, filtered by price or condition, or browsed by category the way a real marketplace can.
- **No intelligent assistance.** Writing a good listing, pricing an item fairly, and finding the right item quickly are all manual, repetitive tasks that AI can meaningfully reduce.

CampusHub solves this by giving students one dedicated, college-verified app that takes them from *discovering* an item to *evaluating* it, *messaging* the seller, *completing* the exchange in person, and leaving a *review* — with AI reducing the effort at almost every step.

1.3 Who Uses CampusHub

User type	What they want	What CampusHub gives them
Student buyer	Wants to find an affordable, relevant item quickly and safely.	Search, filters, natural-language search, recommendations, reviews of the seller.

User type	What they want	What CampusHub gives them
Student seller	Wants to list an item with minimal effort and reach the right buyers.	AI-generated title/description, CV-assisted category detection, price suggestion.
General student	Wants campus info and app help without digging through menus.	RAG-based campus assistant (Part 5.5).
Moderator / Admin	Wants to keep the platform safe and orderly.	Report review dashboard, ability to suspend listings/users.

1.4 What We Are Building First (and What We Are Not)

Build first — the MVP (Weeks 1–9 roughly)

- Student registration/login tied to a college.
- Student profile (course, branch, year).
- Create, edit, delete a marketplace listing, with photos.
- Search and filter listings by category, price, condition.
- Real-time buyer–seller chat.
- AI writes a suggested listing title, description and category.
- Natural-language marketplace search ("find me a calculator under ■700").
- Basic computer-vision category detection from an uploaded photo.
- A simple recommendation list ("you might also like").
- Reviews and reports.

Build after the MVP works — Advanced (Weeks 10–12)

- An AI agent that can search the marketplace on the user's behalf using tools.
- A RAG (retrieval-augmented generation) campus assistant that answers app/campus questions from a fixed knowledge base.
- An AI price assistant that looks at comparable real listings before suggesting a price.
- A smarter, personalized recommendation model.
- Better image moderation, an admin dashboard, analytics, push notifications.

Deliberately out of scope for this release

- Real payments or escrow — CampusHub connects buyer and seller; money changes hands in person, outside the app.
- A full social-media feed.
- Deep integration with the college's official ERP/student information system.
- Blockchain of any kind, unless a very specific future requirement justifies it.
- Training a large foundation model from scratch — we use an existing model (Gemini) via API.

Why this matters: Keeping scope tight is not a limitation — it is what makes it possible for seven students to build, test and demonstrate a genuinely working system in 14 weeks. A smaller system that fully works beats a large system that half-works, every single time in a viva.

1.5 How the Whole System Fits Together, Explained Simply

Think of CampusHub as four separate "departments" that only talk to each other through one receptionist:

- **The Flutter app** (what the student sees and taps) never talks directly to the database or to any AI model. It only ever talks to one place.
- **NestJS (the "Central API")** is that one place — the receptionist. Every request from the app — login, create a listing, send a message, ask the AI to write a description — goes through NestJS first. NestJS checks who you are and whether you're allowed to do what you're asking, then forwards the request to the right department.
- **PostgreSQL** (the database) and **object storage** (R2/S3, for images) hold the actual data. Only NestJS is allowed to talk to them directly.
- **The AI/ML services** (built with FastAPI, in Python) do the actual AI work — calling Gemini for text generation, running the computer-vision model on an image, computing recommendations. NestJS calls these services; the Flutter app never calls them directly, and the AI/ML services never touch the database directly either.

The single most important architecture rule: The one rule that keeps this whole architecture safe and easy to reason about: **nothing talks to the database except NestJS, and nothing talks to the AI models except the AI/ML services, and the only way in for the Flutter app is through NestJS.** If you remember only one architectural fact for the viva, remember this one — it answers most "why did you design it this way" questions.

PART 2

Requirements

Exactly what the product must do, and how good it must be, stated so it can be tested

2.1 Functional Requirements

Each requirement below has an ID so it can be referenced in code comments, commit messages, tickets and test reports (e.g. "implements FR-04").

ID	Module	Requirement (must be demonstrable)	Owner(s)
FR-01	Authentication	Register, log in, log out, access protected features.	M3, M1
FR-02	College identity	Profile stores college, course, branch, year.	M3, M2
FR-03	Profile	View and edit own profile.	M2, M3
FR-04	Create listing	Seller creates a listing: images, title, description, price, category, condition.	M2, M3
FR-05	Manage listing	Seller edits, deletes, marks a listing sold.	M2, M3
FR-06	Search	Search listings by keyword.	M2, M3
FR-07	Filter	Filter by category, price, condition, college.	M2, M3
FR-08	Listing detail	View full listing and seller info.	M2, M3
FR-09	Chat	Real-time buyer/seller messaging.	M2, M3
FR-10	AI generation	AI-generated listing content, editable before save.	M4, M2
FR-11	AI search	Natural-language marketplace search.	M4, M3
FR-12	CV analysis	Uploaded photo analyzed for category hint.	M6, M3
FR-13	Recommendation	System shows relevant listings.	M6, M2
FR-14	Review	Eligible users submit a rating/review.	M3, M2
FR-15	Report	Report a listing or user.	M3, M2
FR-16	Notifications	Key events trigger a notification.	M7, M3
FR-17	AI assistant	Answer supported campus/app questions.	M4
FR-18	Admin moderation	Admin inspects/updates reports.	M3, M7

2.2 Non-Functional Requirements — What "Good" Looks Like

Quality	Requirement	How you prove it (test)
Security	No plaintext passwords anywhere; secrets never committed to Git.	Try to read a password from the database — it must be an unreadable hash.
Performance	Listing and search responses feel instant for normal student use.	Time a search request; it should return well under a second on a normal connection.
Reliability	A failing API call shows an error message, never crashes the app.	Turn off Wi-Fi mid-request and confirm the app recovers gracefully.
Scalability	Services are separable so AI/ML work can scale independently of the API.	AI/ML services can be restarted or scaled without restarting NestJS.
Maintainability	Code organized by feature/module, and documented.	A new member can find where a feature lives within a minute of looking.
Testability	Every major feature has a repeatable test path.	A test for the feature exists and can be re-run on demand.
Observability	Errors and important failures are logged, without secrets.	A log entry shows what failed, never a password/token/API key.
Accessibility	Readable text, clear controls, validation messages, usable contrast.	A person unfamiliar with the app can complete the core flow unaided.
Privacy	Only data necessary for the feature is collected.	For every field on a form, someone can say why it's needed.

PART 3

User Journeys

The exact sequence of screens and actions for every kind of user, spelled out step by step

A "user journey" is simply the ordered list of steps a real person takes to accomplish one goal. Building against a written journey (instead of a vague feature list) keeps the frontend and backend member on the same page about what happens after each tap.

3.1 New Student — First-Time Setup

This is the very first thing a new user experiences. It must work perfectly before any other feature is built on top of it.

1. Install and open the CampusHub app for the first time.
2. Tap Register; enter college email, name and password.
3. Confirm college identity (e.g. verify college email domain).
4. Fill in basic profile: course, branch, year.
5. Land on the Home screen.
6. Browse the marketplace, use search or filters.
7. Open a listing of interest.
8. Start a chat with the seller.

Who builds this step

- **M2** builds the registration/login screens and the profile screens.
- **M3** builds the auth and user APIs, hashes passwords, issues tokens.
- **M1** integrates the auth APIs into the app and manages session state (i.e. keeps the user logged in and knows which screens require login).
- **M6** tests invalid login attempts, unauthorized requests, and expired/invalid tokens.

Product rule: Only once this entire flow works end-to-end should the team move on to building the marketplace. A broken login blocks every other feature from being properly tested.

3.2 Seller — Listing an Item

This is the journey where most of the Generative AI and Computer Vision value shows up.

1. Log in.
2. Tap "Sell Item".
3. Select or take photos of the item.
4. Enter basic information (whatever the seller already knows: rough category, condition).

5 The app runs Computer Vision analysis on the photo and suggests a category.
6 The app asks the AI service to generate a title, description and category suggestion.
7 The seller reviews the AI's suggestion and edits anything they want to change.
8 The seller sets (or accepts the AI-suggested) price.
9 The seller publishes the listing.
10 The seller receives a buyer message.
11 The seller marks the item sold once it is handed over.
12 The seller receives a review from the buyer.

Product rule: AI output is always a suggestion, never an automatic truth. The seller must be able to edit every generated field and must explicitly confirm before anything is published. This single rule is why the UI always shows AI content as editable form fields, never as read-only text that gets silently saved.

3.3 Buyer — Finding and Getting an Item

1 Open Home.
2 Search — either (a) type ordinary keywords, or (b) type a natural sentence like "scientific calculator under 700 in good condition".
3 Review results.
4 Open a listing.
5 Check the seller's profile and past reviews.
6 Start a chat and agree on details.
7 Meet in person and complete the exchange (outside the app — no payment gateway in this release).
8 Leave a review after the transaction.

3.4 AI Agent — Assisted Search (Advanced Feature)

This is what "agentic AI" means in CampusHub: the AI doesn't just answer in text, it takes actions by calling a small, fixed set of backend tools, then reasons over the results.

1 User asks in natural language: "Find a calculator below ■700 in good condition."
2 The agent parses the request into an intent.
3 The agent calls the tool searchListings() with an initial filter.
4 The agent calls filterByPrice() to narrow results to the stated budget.
5 The agent calls filterByCondition() to narrow to the stated condition.
6 The agent calls getUserRating() to check seller reliability before ranking results.
7 The agent ranks and summarizes the results in plain language.
8 The user sees the top relevant results, each backed by a real database record.

Product rule: The agent is never given unrestricted database access. It may only call a short, allowlisted set of read-only backend tools, each with a validated input schema. This is explained fully in Part 5.6.

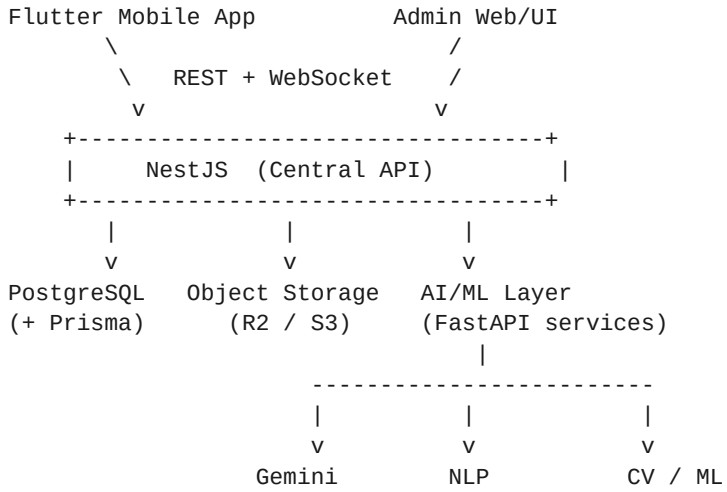
PART 4

System Architecture, Database & API Contract

How the pieces are wired together, what data lives where, and every endpoint the app can call

4.1 High-Level Architecture

Requests flow in one direction, through one gatekeeper. This diagram shows the same idea as Part 1.5, now with the exact technology at each layer:



4.1.1 Data Ownership — Who Is Allowed to Touch What

Component	Owns	Explicitly does NOT own
Flutter	UI, local state, user interactions.	Database credentials, model secrets.
NestJS	Auth, users, listings, chats, permissions, the API contract.	Prompt secrets, model training.
AI service (FastAPI)	LLM calls, prompts, RAG, agent orchestration.	Direct unrestricted database access.
ML service (FastAPI)	Computer-vision inference, recommendation model.	User authorization.
PostgreSQL	Persistent application data.	Images/files.
R2 / S3	Images/files.	Business logic.
Firebase	Push notification delivery.	Marketplace database.

Why this matters: No service reaches past its own boundary. If the Flutter app ever needs data from Postgres, it asks NestJS — it never gets a direct connection string. If the AI service needs a real listing to reason about, NestJS fetches it and hands it over — the AI service never queries the database itself. This containment is what makes the system secure and easy to test in isolation.

4.2 Database Schema, Explained Table by Table

Every table below is a real, persistent record. Read this once before writing any Prisma schema (M3) or any API response type (everyone else).

Table	Key fields	In plain language
colleges	id, name, domain, status	One row per participating college. "domain" lets registration verify a college email address.
users	id, name, email, passwordHash, collegeId, course, branch, year, role	One row per student/admin. passwordHash is never the plaintext password.
categories	id, name, parentId, status	Marketplace categories; parentId allows subcategories (e.g. "Electronics" → "Calculators").
listings	id, sellerId, categoryId, title, description, price, condition, status	One row per item for sale. status covers active/sold/removed.
listing_images	id, listingId, objectKey, url, order	One row per photo; objectKey points at the object storage file.
favorites	id, userId, listingId	A user bookmarking a listing.
chats / chat_members	id, listingId / chatId, userId	A chat is tied to a listing; members are the buyer and seller.
messages	id, chatId, senderId, text, createdAt, readAt	One row per chat message.
reviews	id, reviewerId, reviewedUserId, listingId, rating, text	A rating left after a transaction.
reports	id, reporterId, targetType, targetId, reason, status	A flag raised against a listing or user, for admin review.
notifications	id, userId, type, title, body, readAt	A record of something the user should be told about.
ai_requests	id, userId, feature, inputHash, status, latency	An audit trail of every AI call — which feature, how long it took, whether it succeeded — without storing unnecessary private content.
recommendations	id, userId, listingId, score, reason	A precomputed or on-demand recommendation, with a score and a human-readable reason.

4.3 API Contract — Every Endpoint, What It's For

This is the single agreement between frontend (M1/M2) and backend (M3). No endpoint should be built or consumed without first appearing here.

Group	Endpoint	What it does
Auth	POST /auth/register	Create an account tied to a college.
	POST /auth/login	Exchange credentials for an access/refresh token pair.
	POST /auth/refresh	Exchange a refresh token for a new access token.
	POST /auth/logout	Invalidate the current session.
	GET /auth/me	Return the currently authenticated user.
Users	GET /users/me	Return the caller's own profile.
	PATCH /users/me	Update the caller's own profile.
	GET /users/:id	Return another user's public profile (for viewing a seller).
Listings	POST /listings	Create a listing (seller only).
	GET /listings	Browse listings, optionally filtered.
	GET /listings/:id	Full detail for one listing.
	PATCH /listings/:id	Edit a listing (owner only).
	DELETE /listings/:id	Delete a listing (owner only).
	PATCH /listings/:id/status	Mark a listing sold/active (owner only).
Search	GET /listings/search?q=...	Plain keyword search.
	GET /listings?category=&minPrice=&maxPrice=&condition=	Structured filter search.
Chat	POST /chats	Start a chat tied to a listing.
	GET /chats	List the caller's chats.
	GET /chats/:id/messages	Message history for one chat.
Reviews	POST /reviews	Submit a review.
	GET /users/:id/reviews	Reviews received by a user.
Reports	POST /reports	Report a listing or user.
AI	POST /ai/listing/generate	Get an AI-suggested title/description/category.
	POST /ai/search/parse	Turn a natural-language query into structured filters.
	POST /ai/price/suggest	Get a suggested price range.
	POST /ai/assistant/query	Ask the RAG campus assistant a question.
	POST /ai/agent/search	Run the agentic marketplace search.
ML	POST /ml/image/analyze	Get a category/confidence guess for a photo.
	GET /recommendations	Get the caller's current recommendation list.

4.3.1 The Procedure for Building Every Single Endpoint

M3 (and anyone touching the backend) follows the same 12-step sequence for every endpoint, without exception. This is what prevents the frontend and backend from building against two different mental models of the same feature.

1. Write the requirement in one sentence (what should this endpoint do?).
2. Define the request and response schema (exact field names and types).
3. Create the DTO (Data Transfer Object) matching that schema.
4. Add validation rules to the DTO (required fields, types, ranges).
5. Implement the service method (the actual business logic).
6. Implement the controller (the route that receives the HTTP request).
7. Add the authorization rule (who is allowed to call this, and on whose data).
8. Add the database operation via Prisma.
9. Test the endpoint manually in Postman (happy path + at least one failure path).
10. Document the endpoint (add it to the shared Postman collection / API docs).
11. Integrate it into the Flutter app.
12. Add a regression test so a future change can't silently break it.

PART 5

How Every AI/ML Feature Works

Each feature explained in plain language first, then as a concrete, buildable pipeline

"AI" is not one feature in CampusHub — it is six distinct, narrow features, each with a clear input, a clear output, and a clear owner. Read the plain-English summary of each one even if it isn't your subsystem: every member needs to be able to explain, in the viva, what every AI feature does and why it's built the way it is.

5.1 Listing Generator (Generative AI)

Plain-English summary: The seller gives a few basic facts about the item. The AI turns those facts into a polished title, description and category suggestion — as structured data, not free-flowing prose — which the seller can then edit before anything is saved.

```
Flutter -> NestJS -> AI service -> Gemini -> Structured JSON
   -> NestJS validation -> Flutter preview -> User confirms -> Listing saved
```

Build steps

1. Create a prompt template with explicit fields and constraints (what fields to fill, what not to invent).
2. Send the model only the product information actually needed — nothing extraneous.
3. Request structured output in this exact shape:

```
{
  "suggestedTitle": "...",
  "description": "...",
  "category": "...",
  "keywords": ["..."],
  "warnings": []
}
```

1. Validate the returned structure inside the AI service before it ever reaches NestJS.
2. Reject or repair a malformed response rather than passing it through.
3. Show the result to the seller as editable fields, never as locked-in text.
4. Require explicit user confirmation before the listing is saved.
5. Log latency and the feature name — never log unnecessary private content.

5.2 Natural-Language Marketplace Search (NLP)

Plain-English summary: The user types a normal sentence ("I need a scientific calculator under 700, preferably in good condition"). The AI's only job is to turn that sentence into a small set of structured filters that the database already knows how to use — the AI never invents search results itself.

Build steps

1. Define the allowed filter schema up front (which fields, which value types are legal).

2. Send the user's raw query to the AI service.
3. Force the model's output into that allowed schema — nothing outside it is accepted.
4. Validate numeric fields (e.g. price must be a number) and enum values (e.g. condition must be one of a fixed list).
5. Discard any field the model returned that isn't in the allowed schema.
6. Send only the validated filters to NestJS.
7. NestJS executes a normal parameterized/ORM query — real database listings come back, never AI-invented ones.
8. Optionally, let the AI summarize the results afterward in plain language.

```
Input:  "I need a scientific calculator under 700, preferably in good condition."

Output: {
  "category": "scientific calculator",
  "maxPrice": 700,
  "condition": "good",
  "keywords": ["calculator"]
}
```

5.3 AI Price Assistant

Plain-English summary: The model is never asked to guess a price out of thin air. NestJS first finds real comparable listings from the database; the AI's job is only to read those real numbers and explain a sensible range — the evidence comes from the database, not from the model's imagination.

```
Current item + Comparable listings -> AI price assistant
   -> Suggested range + Short explanation + Confidence/limitations
```

Build steps

1. Query recent listings in the same or a similar category.
2. Filter out invalid or already-sold records where relevant.
3. Compute a simple statistical baseline such as the median price.
4. Send the comparable records to the AI service as evidence.
5. Generate a suggested range with a short plain-language explanation.
6. Display the result to the user clearly as an estimate, not a guarantee.
7. Store the underlying comparison listing IDs if auditability is required later.

5.4 RAG Campus Assistant

Plain-English summary: "RAG" stands for Retrieval-Augmented Generation. In plain terms: instead of trusting the model's general knowledge, we first retrieve the few most relevant paragraphs from a small, fixed knowledge base we control (marketplace rules, FAQ, reporting procedure), then ask the model to answer using only that retrieved text.

```
Question -> Retrieve relevant chunks -> Build context -> Gemini
   -> Answer with context -> Fallback if evidence is missing
```

Build steps

1. Assemble the initial knowledge base: marketplace rules, reporting procedure, user guidelines, CampusHub FAQ, project documentation, and any college information the team and mentor have approved.
2. Chunk that knowledge base into small retrievable pieces.
3. On a question, retrieve the chunks most relevant to it.
4. Build a context block from those chunks and send it to Gemini along with the question.
5. Return the model's answer, grounded in that context.
6. If the knowledge base doesn't contain enough information to answer, say so explicitly — the assistant must never invent an answer it has no evidence for.

5.5 Agentic AI Marketplace Assistant

Plain-English summary: This is the only feature where the AI takes actions, not just generates text. The model is given a small, fixed toolbox — a handful of read-only backend functions — and decides which tools to call, in what order, to satisfy the user's request. It never gets direct database access; every action goes through a validated tool wrapper.

Build steps

1. Define each tool's exact input schema (searchListings, getListing, getUserRating, getCategory, getRecommendations).
2. Implement the corresponding NestJS endpoint/service behind each tool.
3. Create a safe tool wrapper in the AI service that calls that endpoint.
4. Allow only read/search operations in this first version of the agent.
5. Validate every tool argument before it is executed.
6. Run a predefined evaluation set of realistic user requests against the agent.
7. Measure two things separately: did it pick the right tools, and did it produce the right final result.
8. Only later consider allowing write actions, and even then require explicit user confirmation before any consequential action.

Why this matters: An agent that could write to the database directly would be able to take an action a user never explicitly approved. Keeping every tool read-only in this version removes an entire category of risk while the reasoning quality is still being evaluated.

5.6 Computer Vision Pipeline

Plain-English summary: The seller uploads a product photo. A small image-classification model guesses which of a short, realistic category list the item belongs to, with a confidence score. This is a suggestion the seller confirms — never an automatic, silent decision.

```
Image upload -> File validation -> Resize/normalize -> CV model
   -> Class/confidence -> Backend -> AI category mapping -> User confirmation
```

Build steps

1. Define a small, realistic category set (Calculator, Book, Laptop, Keyboard, Mouse, Headphones, Bag, Bottle, Stationery, Other).

2 Collect permitted images or use an appropriate existing dataset.
3 Create a train / validation / test split.
4 Preprocess images (resize, normalize).
5 Establish a simple baseline model before anything fancier.
6 Fine-tune a stronger model only if the baseline clearly isn't good enough.
7 Evaluate on a held-out test set the model never saw during training.
8 Wrap the trained model in a FastAPI inference endpoint.
9 Return the predicted class, its confidence, and the model version.
10 Connect the endpoint into the listing-creation flow.
11 Explicitly test low-confidence images and objects outside the category list.

5.7 Recommendation Engine — Build It in Phases, Not All at Once

Plain-English summary: A good recommendation system is built in increasing layers of sophistication, each one measured before moving to the next. Starting with the fanciest possible model first makes it impossible to tell whether later improvements actually helped.

```
Phase A: Popularity baseline (most-viewed / most-favorited items)
Phase B: Content-based similarity (similar category / similar price band)
Phase C: Personalization (uses the individual user's own signals)
Phase D: Optional collaborative filtering (users like you also liked...)
Phase E: Evaluation and tuning
```

Signals available to use, roughly in order of how easy they are to compute: search terms, viewed categories, favorites, listing freshness, category similarity, price preference where available, general popularity.

PART 6

Team Structure & Individual Execution Plans

One chapter per member — install list, exact setup, week-by-week tasks, tests, Definition of Done

Seven members, seven clearly separated subsystems. No one else should need to touch your code for the system to work — you integrate through the API contract (Part 4.3) and the tool schemas (Part 5.6), not by editing each other's files.

Member	Role	Primary Output
M1	Project / Tech Lead + Flutter Architecture	App architecture, navigation, state, final integration
M2	Flutter UI + Marketplace	All core student-facing marketplace screens
M3	Backend + Database	NestJS, Prisma, PostgreSQL, every API endpoint
M4	Generative AI + NLP + RAG	LLM features, search parser, RAG, prompts
M5	Agentic AI + AI Integration	Tool orchestration, agent evaluation
M6	Computer Vision + ML	CV model, recommendation system, evaluation
M7	Cloud + Security + DevOps + QA	Cloud, storage, notifications, security, CI/CD, testing

Why this matters: Every chapter below follows the same shape on purpose: Role in plain language → what you need to know going in → what to install and why → exact setup commands → folder structure → your week-by-week task list → how you test your own work → your Definition of Done → your dependencies on others → common pitfalls. Skim your teammates' chapters at least once so you know what to expect from them and when.

6.2 MEMBER 1 — Project / Technical Lead + Flutter Architecture

Your job in one sentence: You own the app's skeleton — navigation, state management, API client, error handling — and you are the final integrator who merges everyone else's work into one working demo build.

Helpful to know before you start (not mandatory)

- Basic Dart/Flutter syntax (widgets, async/await).
- What a REST API is (you'll consume many).
- Basic Git (branches, pull requests, merge conflicts).

Install

Install	Why you need it
Git	Version control — everything the team ships goes through Git.
Flutter SDK	The framework the whole app is built in.
Dart SDK	Ships with Flutter automatically — the language Flutter uses.
VS Code	Lightweight editor for day-to-day Flutter work.
Android Studio	Needed for the Android SDK and emulator, even if you write code in VS Code.
Android SDK	Lets you build and run the app on an emulator or device.
Postman	To read and manually test the API contract M3 publishes.

Setup — Run These Commands in Order

```
flutter doctor
flutter create frontend
cd frontend
flutter pub add flutter_riverpod
flutter pub add go_router
flutter pub add dio
flutter pub add image_picker
flutter pub add cached_network_image
```

What each package is for, in plain terms:

- **flutter doctor** — checks your machine is set up correctly before you write a line of code.
- **flutter_riverpod** — how the app remembers state (e.g. "is the user logged in") across screens.
- **go_router** — defines every screen's URL/route and who is allowed to reach it.
- **dio** — the HTTP client every network request to NestJS goes through.
- **image_picker** — lets the seller pick or take a listing photo.
- **cached_network_image** — shows listing images without re-downloading them every time.

Folder Structure You Own

```
lib/
```

```
■■■ core/
■   ■■■ constants/
■   ■■■ errors/
■   ■■■ network/
■   ■■■ utils/
■■■ config/
■■■ routes/
■■■ theme/
■■■ models/
■■■ services/
■■■ providers/
■■■ features/
    ■■■ auth/
    ■■■ home/
    ■■■ marketplace/
    ■■■ search/
    ■■■ chat/
    ■■■ profile/
    ■■■ ai/
```

Your Week-by-Week Task List

Weeks 1-2 — Foundation

1. Run flutter create and commit the empty skeleton to the repo.
2. Set up the folder structure above so every future feature has a home.
3. Build the base theme (colors, typography) used by every screen.
4. Define the full route map in go_router, including which routes require login.
5. Create the Dio HTTP client with the base URL and interceptors (for attaching the auth token automatically, and for catching errors in one place).
6. Agree the model conventions with M3 (how a Listing, User, Message look as Dart classes, matching the API contract exactly).
7. Write the shared API error-mapping logic: every failed request becomes one consistent, user-readable error, never a crash.

Weeks 3-5 — Auth and First Integrations

1. Build the auth/session provider (tracks logged-in state, stores the token securely).
2. Integrate M3's /auth endpoints once they're ready.
3. Integrate M2's login/register/profile screens with real API calls.
4. Confirm protected routes correctly redirect an unauthenticated user to login.

Weeks 6-11 — Ongoing Integration

1. As M2 finishes each marketplace screen, wire it to the real M3 endpoint (replace any mock data).
2. As M4/M5/M6 publish their AI/ML endpoints, integrate their responses into the relevant screens (AI listing preview, natural-language search bar, CV suggestion, recommendations list).
3. Own the final "integration" branch — every feature branch merges into develop, and you are the one who resolves cross-feature conflicts.

4. Review every Flutter pull request before it merges — check it follows the folder/model conventions above.
5. Keep a stable, demo-able build at the end of every week, even if a feature is incomplete (hide it behind a flag rather than shipping something broken).

Weeks 12-14 — Hardening and Demo

1. Fix integration bugs found during the full end-to-end acceptance test (Part 8.5).
2. Generate a release APK and confirm it installs and runs on a clean device.
3. Prepare and rehearse the live demo alongside M7's demo evidence.

How You Test Your Own Work

- Cold-start the app on a clean emulator; confirm navigation works with no leftover state.
- Turn off the network mid-request; confirm the app shows an error, not a crash.
- Log out and try to reach a protected route directly; confirm you're redirected to login.
- Load every major screen using real API data at least once per week, not mock data.

Your Definition of Done

- Navigation works from a clean install.
- API errors do not crash the app.
- Protected routes require authentication.
- All major screens load using real API data.
- A release APK can be generated.

Your Dependencies

You depend on:

- M3's auth/user API contract, published early (Weeks 2-3).
- M2's screens, delivered incrementally per Week 27 build order.
- M4/M5/M6's AI/ML response shapes, matching Part 4.3.

These members depend on you:

- Every member needs your integration branch to demo their feature inside the real app rather than in isolation.

Common Pitfalls

- Building UI polish before the navigation skeleton is solid — fix the skeleton first.
- Letting mock data linger in a screen after the real endpoint is ready — swap it out the same week.
- Merging a teammate's branch without running the app once yourself first.

6.3 MEMBER 2 — Flutter UI + Marketplace

Your job in one sentence: You build every screen a student actually taps through: browsing, searching, creating a listing, chatting, and reviewing AI-generated content.

Helpful to know before you start (not mandatory)

- Basic Flutter widgets and layout (Row, Column, ListView).
- Basic form validation concepts.

Install

Install	Why you need it
Flutter	The framework you build every screen in.
Android Studio	Emulator + Android SDK for running your screens.
VS Code	Day-to-day editing.
Git	Version control.
Postman	To see exactly what shape of data M3's API returns before you build a screen around it.

Your Exact Build Order

1. Build the shared component library first: reusable buttons, text fields, cards, and a loading indicator — every later screen reuses these instead of reinventing them.
2. Build the login/register screens.
3. Build the home screen (entry point after login).
4. Build the category section (browsable list of marketplace categories).
5. Build the listing card (the reusable tile shown in every list/grid of listings).
6. Build the marketplace grid/list screen using that listing card.
7. Build the listing detail screen (full info + seller + reviews + chat entry point).
8. Build the create-listing form (title, description, price, category, condition, photos).
9. Build the edit/delete-listing flow.
10. Build the profile screen (view + edit).
11. Build the search/filter UI (both plain keyword and natural-language modes).
12. Build the AI-generated content preview/edit UI — this is where the seller reviews and edits what M4's listing generator produced.

UI Acceptance Criteria — Check Every Screen Against This List

- Every form has visible validation (required fields, invalid values are caught before submit).
- Every network request shows a loading state and an error state — never a frozen screen.
- Long titles/descriptions never break the layout (test with unusually long text).
- Images have a placeholder while loading and a fallback if they fail to load.
- The user can cancel an AI-generated suggestion entirely.

- The user can edit every AI-generated field before publishing.

Week-by-Week Pacing

- Weeks 1-2: component library + login/register + home skeleton.
- Weeks 3-4: profile, category section, listing card, marketplace grid.
- Weeks 5-6: listing detail, create-listing form (wired to real image upload once M7's storage is ready).
- Week 7: chat screens (paired with M3's realtime endpoints).
- Week 8: AI-generated content preview/edit UI (paired with M4).
- Week 9: natural-language search bar (paired with M4) and CV suggestion UI (paired with M6).
- Weeks 10-11: recommendations UI, edit/delete-listing flow, polish.
- Weeks 12-14: bug fixes found in the full acceptance test, accessibility pass.

How You Test Your Own Work

- Fill every form with empty fields, then with clearly invalid values, and confirm the validation message is specific and visible.
- Put the device in airplane mode and submit a form; confirm a clear error, not a silent failure.
- Paste an unusually long product description into the create-listing form and confirm the layout still looks correct.

Your Dependencies

You depend on:

- M3's listing/user/chat endpoints, matching Part 4.3 exactly.
- M1's component library location and route map.
- M4's AI generation response shape and M6's CV response shape.

These members depend on you:

- M1 needs your screens ready to wire into the integration branch on the weekly cadence above.
- M7 needs your forms finished before writing UI regression tests.

Common Pitfalls

- Building a screen against imagined API data instead of the real contract — always check Part 4.3 or ask M3 first.
- Skipping the loading/error state because "it usually works" in testing.
- Hard-coding a category or condition list that doesn't match what the backend actually accepts.

6.4 MEMBER 3 — Backend + Database

Your job in one sentence: You are the single gatekeeper: every request the app makes, and every AI/ML call, passes through your API, gets checked for permission, and only then touches the database.

Helpful to know before you start (not mandatory)

- Basic TypeScript.
- Basic SQL / relational database concepts (tables, foreign keys).
- What REST endpoints and HTTP status codes mean.

Install

Install	Why you need it
Node.js LTS	Runtime NestJS runs on.
VS Code	Editor.
Git	Version control.
Postman	For manually testing every endpoint you build, per the 12-step procedure (Part 4.3.1).
PostgreSQL (local or managed)	Where all persistent application data actually lives.
NestJS CLI	Scaffolds modules, controllers and services consistently.

Initialize the Project

```
npm i -g @nestjs/cli
nest new backend
cd backend
npm install @prisma/client
npm install -D prisma
npx prisma init
```

Your Build Order

1. Create the database and its environment variables (see Part 7.3 for the exact policy).
2. Write the Prisma schema for every table in Part 4.2.
3. Run the first migration.
4. Build the auth module (register, login, refresh, logout, me) — hash passwords, issue tokens.
5. Build users/colleges.
6. Build categories.
7. Build listings (create/edit/delete/status).
8. Build search/filter.
9. Build chat and messages (REST + WebSocket).
10. Build reviews/reports.
11. Build notification records.

12 Create the AI/ML gateway endpoints — thin NestJS routes that forward validated requests to the FastAPI AI/ML services and return their response.

13 Write API documentation and Postman test collections as you go, not at the end.

Database Migration Discipline

```
Change schema.prisma
  -> npx prisma migrate dev
  -> Review the generated migration file
  -> Test locally
  -> Commit the migration
  -> Everyone else runs: npx prisma migrate deploy (or migrate dev) to pull the latest schema
```

Why this matters: Never ask a teammate to manually edit a production table. Every schema change is a migration file, committed to Git, so the whole team's databases stay in sync and the change history is auditable.

Week-by-Week Pacing

- Week 2: finalize the ERD and API contract with M1/M7 (this becomes Part 4).
- Week 3: project skeleton runs locally; Prisma connects to Postgres.
- Week 4: auth module complete and tested; profile endpoints complete.
- Weeks 5-6: listings CRUD, image-upload endpoint (paired with M7's object storage), search/filter.
- Week 7: chat + messages, including the WebSocket gateway.
- Week 8: AI gateway endpoint for listing generation (paired with M4).
- Week 9: AI gateway endpoints for search-parse and price-suggest.
- Week 10: ML gateway endpoint for image analysis and recommendations (paired with M6).
- Week 11: agent gateway endpoints/tools (paired with M5) — reviews, reports, notifications.
- Week 12: security hardening pass against the Part 9 checklist.
- Weeks 13-14: bug fixing from the full acceptance test; freeze the API for release.

How You Test Your Own Work

- For every endpoint: one Postman test for the happy path, one for invalid input, one for unauthorized access, one for a missing resource (Part 8.2).
- Confirm a password is never returned in any API response, ever, including nested objects.
- Confirm every mutation endpoint checks ownership or role before making a change.

Your Dependencies

You depend on:

- M7's object storage endpoint conventions for image URLs.
- M4/M5/M6's FastAPI service contracts (request/response shape) to build the gateway correctly.

These members depend on you:

- M1 and M2 cannot build real screens without your endpoints matching Part 4.3.
- M4, M5, M6 all call into your listing/user/search data through their tools.

Common Pitfalls

- Changing a response field name after M1/M2 have already integrated it — always update Part 4.3 and tell the team the same day.
- Skipping authorization checks on an endpoint because "it's just for testing".
- Letting the AI gateway forward an unvalidated request straight to the AI service.

6.5 MEMBER 4 — Generative AI + NLP + RAG

Your job in one sentence: You turn a handful of facts, or a plain sentence, into structured, validated AI output that the rest of the app can trust and render — never raw, unchecked model text.

Helpful to know before you start (not mandatory)

- Basic Python.
- What an LLM prompt is.
- The idea of "structured output" (asking a model to return JSON matching a schema).

Install

Install	Why you need it
Python	Language the AI service is written in.
VS Code	Editor.
Git	Version control.
Postman	For testing your FastAPI endpoints directly, before NestJS's gateway calls them.

Setup

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install fastapi uvicorn pydantic python-dotenv
pip install google-genai
```

Your Build Order

1. Create the FastAPI service and a /health endpoint (so M3 and M7 can confirm it's alive).
2. Create the Gemini client, reading the API key only from an environment variable.
3. Build the listing-generation prompt and its structured-output validation (Part 5.1).
4. Build the natural-language search parser and its schema validation (Part 5.2).
5. Build the price-assistant service, consuming comparable listings supplied by NestJS (Part 5.3).
6. Design the knowledge-base ingestion format for the RAG assistant.
7. Build RAG retrieval over that knowledge base.
8. Build the campus assistant endpoint that ties retrieval + Gemini together (Part 5.4).
9. Build an AI evaluation dataset: 30-50 representative queries with the expected correct output, used every time a prompt changes.

Prompt Engineering Rules You Always Follow

- Give the model exactly one well-defined task per prompt — never bundle two jobs into one call.
- Always specify the exact output schema in the prompt.
- Always specify the allowed categories/enums explicitly — never let the model invent a new one.

- Explicitly instruct the model not to invent facts it wasn't given.
- Keep application evidence (real data) clearly separate from model-generated prose in your prompt template.
- Validate model output in code before it is ever returned — never trust it blindly.
- Version your prompts (e.g. in a file with a version number or Git history) so a change in behaviour is always explainable and reproducible.

Week-by-Week Pacing

- Weeks 2-3: FastAPI skeleton, /health endpoint, Gemini client working end-to-end on a trivial prompt.
- Week 8: listing-generation feature complete, validated, integrated with M3's gateway and M2's preview UI.
- Week 9: natural-language search parser and price assistant complete.
- Week 11: RAG knowledge base assembled and the campus assistant answering correctly, including the "I don't have enough information" fallback path.
- Weeks 12-14: run the evaluation dataset against every feature, fix regressions, document results for the AI report (Part 12).

How You Test Your Own Work

- Run your 30-50 query evaluation set after any prompt change; record accuracy before/after.
- Deliberately send a malformed or missing field; confirm your validation rejects or repairs it rather than passing it downstream.
- Ask the campus assistant something outside the knowledge base; confirm it says so instead of making something up.

Your Dependencies

You depend on:

- M3's gateway endpoints to receive real requests and return your output.
- M2's preview UI, so you know exactly what fields it expects.

These members depend on you:

- M2 needs your exact response shape before building the preview/edit UI.
- M5 reuses your prompt-validation patterns for the agent's own outputs.

Common Pitfalls

- Returning free-form prose where a structured field was promised.
- Forgetting to re-run the evaluation set after a prompt tweak — silent regressions are the most common AI bug in student projects.
- Letting the RAG assistant answer from general model knowledge instead of the retrieved context.

6.6 MEMBER 5 — Agentic AI + AI Integration

Your job in one sentence: You build the agent that reasons over a fixed, safe toolbox of read-only backend functions to satisfy a user's request — and you own the shared FastAPI/Gemini integration patterns M4 and you both rely on.

Helpful to know before you start (not mandatory)

- The listing-generation and prompt-validation patterns from Part 5.1 and Part 5.4.
- Basic Python and async programming.

Install

Install	Why you need it
Python	Language the agent service is written in.
VS Code	Editor.
Git	Version control.
Postman	For testing tool endpoints directly.

Setup

```
python -m venv .venv
# Windows: .venv\Scripts\activate
pip install fastapi uvicorn pydantic python-dotenv
pip install google-genai
```

Your Exact Tasks

1. Define the agent's allowed tools: searchListings, getListing, getUserRating, getCategory, getRecommendations.
2. Create a strict input schema for each tool.
3. Connect each tool wrapper to the corresponding M3 API endpoint.
4. Implement the search agent's reasoning loop (parse request → decide which tools to call → call them → rank/summarize).
5. Implement the result-ranking/summarization step.
6. Add validation on every tool call's arguments before executing it.
7. Create a set of agent test cases covering realistic and edge-case requests.
8. Measure tool-selection success (did it call the right tools?) separately from task-completion success (was the final answer actually correct?).
9. Handle tool failures and timeouts gracefully — the agent must never hang or crash the request.
10. Integrate the agent's UI hooks with M1/M2 (loading state while the agent works, showing which step it's on if useful).

Agent Safety Rules — Non-Negotiable

- Read-only tools only, in this version.
- No direct database credentials are ever given to the AI service.
- Every tool input is validated before the tool executes.
- A timeout and a maximum number of tool calls per request are enforced.
- Every tool call is logged with its name and status (never its raw arguments if they contain personal data).
- Any future consequential action (anything that writes data) requires an explicit user confirmation step — never auto-executed.

Week-by-Week Pacing

- Weeks 2-3: FastAPI skeleton shared/aligned with M4's service conventions.
- Weeks 9-10: tool wrappers built and individually tested against M3's real endpoints.
- Week 11: full agent reasoning loop working end-to-end against the predefined scenario set.
- Week 12: safety hardening — timeouts, max tool calls, failure handling.
- Weeks 13-14: evaluation metrics finalized for the AI report; UI integration polish.

How You Test Your Own Work

- Run the full predefined scenario set and record tool-selection accuracy and task-completion accuracy separately.
- Deliberately make one tool fail (e.g. point it at a wrong URL temporarily) and confirm the agent fails gracefully rather than hanging.
- Send a request requiring more tool calls than the configured maximum and confirm it stops safely.

Your Dependencies

You depend on:

- M3's read endpoints being stable before you can build reliable tool wrappers.
- M4's prompt-validation conventions, which you reuse for the agent's own outputs.

These members depend on you:

- M1/M2 need your agent's response shape to build the assistant UI.

Common Pitfalls

- Wiring a tool directly to a write endpoint "just to test it" — this breaks the read-only rule even temporarily.
- Not capping tool calls, which can turn one user request into an unbounded loop.
- Measuring only whether an answer looks plausible instead of checking it against the real database.

6.7 MEMBER 6 — Computer Vision + ML

Your job in one sentence: You build the model that looks at a product photo and guesses its category, and the recommendation system that decides which listings to surface to each user — both measured against a held-out test set, not just eyeballed.

Helpful to know before you start (not mandatory)

- Basic Python and NumPy/Pandas.
- The difference between training, validation and test data.
- Basic classification metrics (accuracy, precision, recall).

Install

Install	Why you need it
Python	Language for all model code.
VS Code	Editor.
Git	Version control.
Jupyter	For exploratory data work and quick experiments.
Postman	For testing your inference endpoint directly.

Setup

```
python -m venv .venv
# Windows: .venv\Scripts\activate
pip install numpy pandas pillow opencv-python
pip install scikit-learn
pip install torch torchvision
pip install ultralytics
pip install fastapi uvicorn
```

Your Exact Tasks

1. Define the category set (Part 5.6): Calculator, Book, Laptop, Keyboard, Mouse, Headphones, Bag, Bottle, Stationery, Other.
2. Prepare a dataset — collect permitted images or use an appropriate public dataset.
3. Write a preprocessing script (resize, normalize).
4. Create a train / validation / test split.
5. Build a simple baseline classifier first.
6. Fine-tune a stronger model only if the baseline is clearly insufficient.
7. Evaluate on the held-out test set and record accuracy/precision/recall/F1.
8. Save the model with a clear version identifier.
9. Build the FastAPI inference endpoint.
10. Return the predicted class, confidence, and model version from the endpoint.
11. Connect the endpoint to the listing-creation flow (paired with M2/M3).

12 Explicitly test low-confidence and out-of-category images.
13 Build the recommendation baseline (Phase A in Part 5.7).
14 Add personalization signals (Phase B/C).
15 Evaluate recommendation quality with Precision@K / Recall@K.
16 Document every experiment: what changed, and what the metric was before and after.

Week-by-Week Pacing

- Weeks 2-4: dataset collected and split; preprocessing script working.
- Weeks 5-6: baseline CV classifier trained and evaluated on the test set.
- Week 10: CV inference endpoint live and connected to listing creation; recommendation baseline (popularity) live.
- Week 11: content-based and light personalization signals added to recommendations.
- Weeks 12-14: evaluation and tuning pass on both CV and recommendation systems; write up metrics for the ML/CV report.

How You Test Your Own Work

- Never judge the model only on images it was trained on — always report the held-out test set score.
- Feed the model a photo of something outside your category list and confirm it degrades sensibly (low confidence, or an "Other" classification) instead of a confident wrong answer.
- Recompute Precision@K/Recall@K any time you change a recommendation signal, and keep the before/after numbers.

Your Dependencies

You depend on:

- M3's listing/category data model, and M7's object storage for the actual image files.
- M2's upload flow to know exactly what image format/size arrives at your endpoint.

These members depend on you:

- M2 needs your response shape (class, confidence, model version) for the CV-suggestion UI.
- M3 needs your recommendation output shape for the /recommendations endpoint.

Common Pitfalls

- Reporting training accuracy instead of test accuracy — this is the single most common mistake examiners look for.
- Jumping straight to a complex recommender without a measured baseline to compare against.
- Not versioning saved models, making a reported metric impossible to reproduce later.

6.8 MEMBER 7 — Cloud + Security + DevOps + QA

Your job in one sentence: You are the safety net for the whole project: cloud infrastructure, storage, notifications, the security checklist, CI, and the test collections that prove every other member's feature actually works.

Helpful to know before you start (not mandatory)

- Basic Docker concepts (image, container).
- Basic idea of environment variables and secrets.
- Basic idea of CI (running checks automatically on every push).

Install

Install	Why you need it
Git	Version control.
VS Code	Editor.
Docker Desktop	To containerize and run services consistently across machines.
Postman	For the shared regression collection you own.
Firebase CLI	For configuring push notifications.

Your Exact Tasks

1. Create the cloud project accounts the team will use.
2. Set up the object storage bucket (R2/S3) and agree naming conventions with M2/M3.
3. Configure the Firebase project and push notifications.
4. Create the environment variable templates (Part 7.3) for every service.
5. Write Dockerfiles for the backend and each AI/ML service.
6. Write a docker-compose file for local integration testing where useful.
7. Configure CI checks (Part 10) so pushes to develop are automatically checked.
8. Create and maintain the security checklist (Part 9) and audit the team against it regularly.
9. Create and maintain the shared API regression Postman collection.
10. Create the end-to-end test matrix (Part 8) covering every core flow.
11. Track bugs in GitHub Issues with clear reproduction steps.
12. Prepare the release checklist and deployment documentation (Part 10, Part 12).

Security Checklist You Own (see Part 9 for full detail)

- Passwords hashed, never stored in plaintext.
- JWT-protected endpoints enforced.
- Admin routes are role-protected.
- File type/size validated on every upload.

- Rate limits applied to sensitive endpoints.
- CORS restricted to known origins.
- No secrets in the repository, ever.
- No sensitive information appears in logs.
- AI tool permissions are limited to read-only (Part 5.6).
- Database credentials are never exposed to the Flutter client.

Week-by-Week Pacing

- Week 1: accounts and repository set up; issue board created.
- Week 2: environment variable templates agreed with M3/M4/M5/M6.
- Week 3: Dockerfiles and docker-compose working for a fully local run of every service.
- Weeks 4-11: continuously growing the Postman regression collection as each new endpoint ships; running the security checklist against new features as they land.
- Week 6: object storage and Firebase fully configured.
- Week 12: full security-checklist pass before hardening freeze.
- Week 13: full end-to-end acceptance test executed and evidenced.
- Week 14: production deployment executed and smoke-tested; release checklist signed off.

How You Test Your Own Work (and Everyone Else's)

- Every new endpoint gets at least the four Postman test cases from Part 8.2 added to the shared regression collection the same week it ships.
- Run the full end-to-end acceptance test (Part 8.5) on a clean environment, not a developer's already-configured machine.
- Attempt each item on the security checklist as an attack, not just a checkbox — e.g. actually try to call an admin route as a normal user.

Your Dependencies

You depend on:

- Every member's endpoints and features being feature-complete enough to test — you cannot write a regression test for a feature that doesn't exist yet.

These members depend on you:

- Everyone relies on your CI, your regression suite, and your security checklist to catch problems before the demo.

Common Pitfalls

- Writing tests only for the happy path — the failure paths are usually where real bugs hide.
- Treating the security checklist as a one-time task instead of a recurring audit as new features land.
- Deploying for the first time in Week 14 instead of rehearsing the deployment sequence earlier.

PART 7

Shared Team Practices

Git workflow, accounts, environment variables and commit conventions — read once as a team

7.1 Git — Set Up and Rules

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git clone <TEAM_REPOSITORY_URL>
cd CampusHub
```

GitHub Rules Everyone Follows

- **main** is the release/stable branch — it should always be demo-ready.
- **develop** is the integration branch — daily work merges here first.
- Every feature is built on its own feature branch, never directly on develop or main.
- Pull Requests are mandatory, even for small changes.
- At least one other member reviews every Pull Request before it merges.
- Commit messages describe the change, not just "fix" or "update".
- Never commit a .env file or any secret value.

Suggested Commit Format

```
feat: add listing creation API
fix: handle expired JWT
ui: add listing detail screen
ai: add structured listing generator
ml: add calculator classifier
test: add auth regression tests
docs: update database schema
```

7.2 Required Accounts and Reference Sites

Purpose	Website	Used by
Source control	https://github.com/	All
Flutter	https://docs.flutter.dev/	M1, M2
Android development	https://developer.android.com/studio	M1, M2
NestJS	https://docs.nestjs.com/	M3
PostgreSQL	https://www.postgresql.org/	M3, M7

Purpose	Website	Used by
Prisma	https://www.prisma.io/docs/	M3
Gemini API	https://ai.google.dev/gemini-api/docs/	M4, M5
Firebase	https://firebase.google.com/docs/flutter/setup	M1, M2, M7
Cloudflare R2	https://developers.cloudflare.com/r2/	M7
PyTorch	https://pytorch.org/	M6
Ultralytics	https://docs.ultralytics.com/	M6
Postman	https://www.postman.com/downloads/	All
VS Code	https://code.visualstudio.com/download	All

7.3 Environment Variable Policy

Every service reads secrets only from environment variables, never from hard-coded values in source code.

```
.env
DATABASE_URL=
JWT_SECRET=
GEMINI_API_KEY=
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=
FIREBASE_PROJECT_ID=
```

- Commit a .env.example file with empty placeholder names, so teammates know what to fill in.
- Never commit real values — this is the single most common accidental security leak in student projects.
- Use separate development and production secrets.
- Rotate credentials immediately if they are ever accidentally exposed (e.g. pushed to Git by mistake).

PART 8

Testing Strategy

What to test, at every layer, so a passing demo is never a lucky accident

8.1 Unit Testing

- Backend service logic (M3).
- Validation rules (M3, M4, M5).
- AI response parsing (M4, M5).
- Recommendation calculations (M6).
- Shared utility functions (all members).

8.2 API Testing — Four Cases per Endpoint, Minimum

1. The success case — correct input produces the expected result.
2. Invalid input — malformed or missing fields are rejected with a clear error.
3. Unauthorized access — a request without a valid token, or the wrong user, is rejected.
4. Missing resource — requesting something that doesn't exist returns a clean not-found error, not a crash.

Sensitive or abuse-prone endpoints (login, AI endpoints, report submission) additionally get a rate/abuse test.

8.3 UI Testing

- Login flow.
- Create listing.
- Search/filter.
- Open listing.
- Chat.
- AI generation preview/edit.
- Profile.

8.4 AI and ML Evaluation

AI evaluation (M4, M5)

- Maintain 30-50 representative natural-language search queries with their expected structured filters.
- Measure exact and partial field accuracy against that expected set.

- Maintain a listing-generation quality rubric (does it read naturally? does it stay factual to the input?).
- Explicitly test malformed or ambiguous requests.
- Explicitly test for hallucination and confirm the fallback ('I don't have enough information') behaves correctly.

ML evaluation (M6)

- Maintain a held-out test set that is never used for training or tuning.
- Report accuracy/precision/recall/F1 for the CV classifier.
- Report Precision@K/Recall@K for the recommendation system.
- Record the model version and experiment settings alongside every reported metric.

8.5 The Full End-to-End Acceptance Test

The release candidate is not ready until this entire flow can be demonstrated, in order, on a clean environment:

1. Register student
2. Log in
3. Complete profile
4. Create listing
5. Upload image
6. Run CV analysis
7. Generate AI description
8. Edit and publish listing
9. Search listing normally
10. Search using natural language
11. Open listing
12. Start chat
13. Send message
14. Mark listing sold
15. Submit review
16. Submit a report on a test item
17. Receive notification
18. Admin reviews report

PART 9

Security Requirements

The non-negotiable rules that protect users, credentials and the platform

Area	Requirement
Passwords	Hash with a strong password-hashing algorithm; never store plaintext.
Authentication	Short-lived access token plus a controlled refresh strategy.
Authorization	Verify ownership/role on every protected mutation, every time.
Uploads	Validate MIME type, extension, size and storage path.
Secrets	Environment variables or a secret manager — never Git.
AI	Allowlisted tools only, validated parameters, no direct database credentials.
Database	Use the ORM/parameterized queries; restrict network access to the database.
API	Validate every DTO; rate-limit sensitive endpoints.
Logs	Never log passwords, tokens or API keys.
Privacy	Collect only the information a feature actually needs.

Why this matters: M7 owns this checklist, but every member is responsible for their own subsystem passing it. Run through it again whenever a new feature is added, not only at the end of the project.

PART 10

Deployment Plan

How the system moves from a laptop to something a real user could open

10.1 Development Environment

```
Developer laptop -> GitHub -> Local services
                              ■■■ Flutter
                              ■■■ NestJS
                              ■■■ PostgreSQL
                              ■■■ AI service
                              ■■■ ML service
```

10.2 Production Concept

```
Flutter APK -> HTTPS -> Cloud-hosted NestJS
                        ■■■ Managed PostgreSQL
                        ■■■ Cloudflare R2
                        ■■■ AI service
                        ■■■ ML service
                        ■■■ Firebase notifications
```

10.3 Deployment Sequence

1. Freeze the release branch.
2. Run automated checks (CI).
3. Build backend containers.
4. Configure production environment variables.
5. Run database migrations.
6. Deploy the backend.
7. Deploy the AI/ML services.
8. Configure object storage.
9. Configure Firebase.
10. Build the Flutter release (APK).
11. Run smoke tests against the live deployment.
12. Record the deployment version.

Why this matters: Rehearse this sequence at least once before Week 14 — the first time anyone runs a deployment sequence should never be during the final release week.

PART 11

14-Week Timeline & Weekly Routine

The milestone schedule, the weekly cadence, and what evidence to keep as you go

11.1 The Fourteen-Week Plan

Week	Milestone	Exit condition
1	Problem validation + PRD + GitHub	Approved scope and issue board
2	Architecture + database + API contract	ERD and API draft frozen
3	All environments + skeleton apps	All services run locally
4	Authentication + profile	Register/login/profile works end-to-end
5	Marketplace backend + UI	CRUD listing flow works
6	Image storage + search/filter	Search and image upload work
7	Chat	Buyer/seller chat works
8	AI listing generator	AI output is structured and editable
9	Natural-language search + price assistant	Real DB results from parsed requests
10	CV model + recommendation baseline	Model endpoint and baseline metrics exist
11	Agentic AI + RAG	Agent completes predefined scenarios
12	Notifications + security hardening	Security checklist passes
13	Full integration + bug fixing	End-to-end acceptance test passes
14	Deployment + report + demo	Release candidate and presentation ready

11.2 Weekly Operating Routine

- **Monday:** assign issues and confirm cross-member dependencies for the week.
- **During the week:** work on feature branches only.
- **Mid-week:** each member posts a short progress update on the issue board.
- **Before merging:** the reviewer checks functionality and tests, not just that it compiles.
- **End of week:** build the integration branch and demo the completed milestone to the team.
- **End of week:** update documentation while the implementation is still fresh.

11.3 Dependency Map

```
M3 Backend
  ■■■ required by M1/M2 frontend integration
  ■■■ required by M4 AI integration
  ■■■ required by M5 agent tools
  ■■■ required by M6 testing
M4 AI      -> requires stable listing/user/search APIs from M3
M5 Agent   -> requires M3 tool endpoints + M4 AI infrastructure
M6 CV/ML   -> requires image contract + listing categories
M7 Cloud/Security -> works continuously across all modules
```

Why this matters: Dependencies live in GitHub Issues, not in anyone's memory. If a task is blocked, mark it blocked immediately instead of silently waiting for someone else.

11.4 GitHub Project Board Columns

```
BACKLOG -> READY -> IN PROGRESS -> BLOCKED -> CODE REVIEW -> TESTING -> DONE
```

Issue Template Fields

- Title.
- Problem/goal.
- Acceptance criteria.
- Owner.
- Priority.
- Dependencies.
- Test plan.
- Evidence required.

11.5 Definition of Done (Applies to Every Task, for Every Member)

- Requirement is implemented.
- The happy path works.
- At least one failure path is tested.
- No hard-coded secrets.
- Code follows the module's conventions.
- Documentation is updated.
- Unit/API/UI tests appropriate to the feature exist.
- Pull Request is reviewed.
- Feature works against the current develop branch.
- Demo evidence or a screenshot is stored where useful.

11.6 What to Record as You Go (Evidence Log)

- Architecture diagrams.
- Database ERD versions.
- API screenshots/Postman collections.
- AI prompt versions.
- AI evaluation examples.
- CV dataset information and license.
- Training metrics.
- Recommendation metrics.
- Deployment screenshots.
- GitHub contribution history.
- Bug reports and fixes.
- Major design decisions and the reasons behind them.

Why this matters: This evidence is what makes individual contribution and technical depth demonstrable during evaluation — collect it weekly, not the night before the demo.

PART 12

Final Deliverables & Acceptance Checklist

Everything that must exist by Week 14, and who is accountable for each item

12.1 Final Academic Deliverables

Deliverable	Owner	Supporting members
PRD	M1	All
System architecture	M1	M3, M7
Database ERD	M3	M1
API documentation	M3	M1, M7
Flutter application	M1/M2	M3
AI report	M4/M5	M1
ML/CV report	M6	M4
Cloud/security report	M7	M3
Test report	M7	All
Final presentation	M1	All
Demo video/screenshots	M1/M7	All
Project report	All	M1 coordinates

12.2 Final Project Acceptance Checklist

- All seven members have clearly documented ownership.
- GitHub repository has clean branch history.
- README contains setup instructions.
- Flutter app runs on a clean machine.
- Backend starts from documented commands.
- Database migration works from a clean environment.
- Authentication works.
- Marketplace works.
- Image upload works.
- Search/filter works.
- Chat works.
- Generative AI works with validated output.

- Natural-language search returns real database results.
- CV service returns model output and confidence.
- Recommendation baseline works.
- Agentic workflow completes predefined scenarios.
- RAG assistant answers supported questions.
- Security checklist passes.
- End-to-end acceptance test passes.
- Deployment works.
- Project report is complete.
- Presentation is complete.
- Each member can explain their own contribution.

12.3 Official Reference Sites

- Flutter: https://docs.flutter.dev/
- Dart: https://dart.dev/learn
- Android Studio: https://developer.android.com/studio
- NestJS: https://docs.nestjs.com/
- PostgreSQL: https://www.postgresql.org/docs/
- Prisma: https://www.prisma.io/docs/
- Gemini API: https://ai.google.dev/gemini-api/docs/
- Firebase Flutter setup: https://firebase.google.com/docs/flutter/setup
- Cloudflare R2: https://developers.cloudflare.com/r2/
- PyTorch: https://pytorch.org/
- Ultralytics: https://docs.ultralytics.com/
- GitHub: https://github.com/
- GitHub Actions: https://docs.github.com/en/actions
- Postman: https://www.postman.com/downloads/
- VS Code: https://code.visualstudio.com/download

12.4 Final Execution Principle

Build in dependency order, not personal preference. Establish the repository and architecture first. Then authentication and data contracts. Then the marketplace. Then chat. Then AI and ML. Then agentic workflows. Finally harden security, test the entire system, and deploy. The goal is not to maximize the number of technologies listed in the report — it is one coherent, working system, in which every technology has a clear purpose, every member owns a measurable subsystem, and every major claim can be demonstrated at the final assessment.

PART 13

Viva Preparation

Every expected question, with a model answer written in the same plain language as this document

Read your own role's section closely enough to answer confidently without notes. Skim every other section — in a group viva you may be asked to explain a teammate's subsystem at a high level, and understanding the whole system (Part 1) makes that easy.

13.1 Member 1 — Project / Tech Lead + Flutter Architecture

Why Flutter?

One codebase targets Android and iOS together, which matters for a 7-person team with limited time; it also has a large widget ecosystem for building forms, lists and chat UIs quickly.

Why Riverpod?

It gives predictable, testable state management without the boilerplate of some older approaches, and it plays well with async data — most of our state (auth, listings) arrives from a network call.

How does navigation work?

go_router defines every screen as a named route, including which routes require an authenticated session; unauthenticated access to a protected route redirects to login.

How does the app handle API errors?

Every request goes through one Dio client with a shared interceptor; any failure is mapped to one consistent error type the UI knows how to display, so a network failure never crashes the screen.

Why is NestJS between Flutter and other services?

So the Flutter app, the database, and the AI/ML services never talk directly to each other. NestJS is the one place that checks who's asking and what they're allowed to do, which makes the whole system far easier to secure, test and reason about.

13.2 Member 2 — Flutter UI + Marketplace

How does listing creation work?

The seller fills basic info and photos; the app calls the CV endpoint and the AI generation endpoint, shows both results as editable fields, and only saves the listing once the seller explicitly confirms.

How is image upload handled?

The image is sent to object storage (via the backend), which returns a URL; that URL, not the raw image, is what's stored against the listing.

How is AI output made editable?

AI-generated fields are rendered in the same form widgets as manually entered ones — there's no separate read-only "AI text" state, so editing an AI suggestion is identical to editing anything else on the form.

How are loading/error/empty states implemented?

Every screen that calls the network has three explicit UI states in addition to the loaded state, and each screen is tested in all three deliberately, not just the happy path.

13.3 Member 3 — Backend + Database

Why NestJS?

It gives a structured, modular way to organize controllers/services/DTOs in TypeScript, with built-in support for validation and guards, which keeps a 7-person backend consistent.

Why PostgreSQL?

It's a mature, relational database well suited to the clearly related entities in this system (users, listings, chats, reviews) and has strong tooling support via Prisma.

Why Prisma?

It gives type-safe database access and a migration workflow the whole team can follow without hand-writing SQL for every change.

Explain JWT.

A JSON Web Token is a signed, tamper-evident token issued at login; the client sends it on every subsequent request, and the server verifies its signature and expiry instead of looking up a session in the database on every call.

Explain database relationships.

For example, one user can own many listings (one-to-many); a chat has many members through a join table (many-to-many); a listing has many images (one-to-many). Foreign keys enforce these relationships at the database level.

Explain REST and WebSockets.

REST is request/response — the client asks, the server answers, then the connection is done; we use it for almost everything. WebSockets keep a connection open so the server can push new chat messages to the client immediately, which REST can't do efficiently.

How do migrations work?

A schema change is written in Prisma's schema file, then `prisma migrate dev` generates a versioned SQL migration file; that file is committed to Git so every teammate's database can be brought to the same state by running the migration.

13.4 Member 4 — Generative AI + NLP + RAG

What is Generative AI?

A model that produces new content (text, in our case) conditioned on an input prompt, rather than simply classifying or retrieving existing content.

How does structured output work?

The prompt explicitly specifies the exact JSON shape the answer must take; the response is then parsed and validated in code, and rejected or repaired if it doesn't match — the model's freedom is deliberately constrained.

What is NLP?

Natural Language Processing — here, specifically, turning a user's plain sentence into a small set of structured search filters the database already understands.

What is RAG?

Retrieval-Augmented Generation: instead of relying on the model's general knowledge, we retrieve the most relevant pieces of our own fixed knowledge base first, then ask the model to answer using only that retrieved context.

How do you reduce hallucinations?

By constraining output to a schema, by supplying real retrieved evidence for the model to ground its answer in, by explicitly instructing it not to invent unavailable facts, and by having it say 'I don't have enough information' when the knowledge base doesn't cover the question.

How does natural-language search become a database query?

The model's raw output is forced into a fixed filter schema, invalid or out-of-schema fields are discarded, and only the validated filters are ever passed to a normal parameterized database query — the model never generates the results, it only generates the filters.

13.5 Member 5 — Agentic AI + AI Integration

What is an AI agent?

A system where the model doesn't just produce text — it decides which of a fixed set of tools/functions to call, in what order, based on the user's request, and reasons over the results.

What is a tool call?

A single, well-defined function the agent is allowed to invoke — for example searchListings(filters) — with a strict input schema that's validated before execution.

Why does the agent not access the database directly?

So its access can be limited to a small, auditable, read-only set of operations instead of an open-ended connection that could be misused, whether by a flawed reasoning step or a malicious input.

How do you evaluate an agent?

By running a predefined set of realistic requests and measuring two things separately: whether it selected the right tools, and whether the final answer was actually correct against the real database.

What happens when a tool fails?

The call times out or errors are caught explicitly; the agent reports a graceful failure to the user instead of hanging or crashing the request.

Why require confirmation for consequential actions?

Because an action that changes data should only ever happen with the user's explicit, informed approval — this keeps a reasoning mistake from turning into a real, unwanted change.

13.6 Member 6 — Computer Vision + ML

What is Computer Vision?

The field of getting a model to extract information — here, a category label — from an image.

Classification vs object detection?

Classification assigns one label to an entire image (what we use — 'this photo is a calculator'); object detection additionally locates where in the image each object is, which we don't need for this feature.

What dataset did you use?

State the dataset/source actually used, its size, and its license, plus the train/validation/test split sizes.

What are precision/recall/F1?

Precision is how many of the model's positive predictions were actually correct; recall is how many of the actual positives the model successfully found; F1 is their harmonic mean, useful when you care about both.

How does recommendation work?

It starts from a popularity baseline, then adds content-based similarity, then personalization using the individual user's own signals, each phase measured before moving to the next.

Why use a baseline before a complex model?

Without a baseline there's no way to know whether a more complex model is actually an improvement — the baseline is what every later claim of 'better' gets measured against.

13.7 Member 7 — Cloud + Security + DevOps + QA

What is cloud computing?

Using infrastructure — databases, storage, compute — hosted and managed by a provider instead of on a local machine, so the system can run reliably outside any one developer's laptop.

Why object storage?

Images are large, unstructured binary files; storing them in PostgreSQL directly would bloat the database, so we keep files in object storage (R2/S3) and only store their reference URL in the database.

How are secrets protected?

They live only in environment variables or a secret manager, are never committed to Git, and are rotated immediately if ever exposed.

What is CI/CD?

Continuous Integration automatically runs checks (build, lint, tests) on every push; Continuous Deployment (where used) automatically ships a passing build. We use CI to catch regressions before they reach develop.

How did you test the system?

Unit tests for logic, a four-case Postman suite per endpoint, UI tests for every core screen, AI/ML evaluation against held-out data, and a full end-to-end acceptance test on a clean environment before release.

What security issues were considered?

Password storage, token handling, authorization on every mutation, upload validation, rate limiting, CORS, secret handling, log hygiene, and keeping the AI agent's permissions read-only and allowlisted.